

Apache Kafka - Complete Reference Guide

Table of Contents

1. [Core Concepts](#)
2. [Kafka Architecture](#)
3. [Topics and Partitions](#)
4. [Producers - Deep Dive](#)
5. [Consumers - Deep Dive](#)
6. [Consumer Groups](#)
7. [Message Queue Patterns](#)
8. [Replication and Fault Tolerance](#)
9. [Kafka CLI Commands Cheatsheet](#)
10. [Performance Tuning](#)
11. [Best Practices](#)
12. [Troubleshooting Guide](#)

Core Concepts

What is Apache Kafka?

Apache Kafka is a **distributed event streaming platform** designed for:

High-throughput, fault-tolerant message publishing
Real-time data pipelines and streaming applications
Storing streams of records in a fault-tolerant, durable way
Processing streams of records as they occur

Key Characteristics

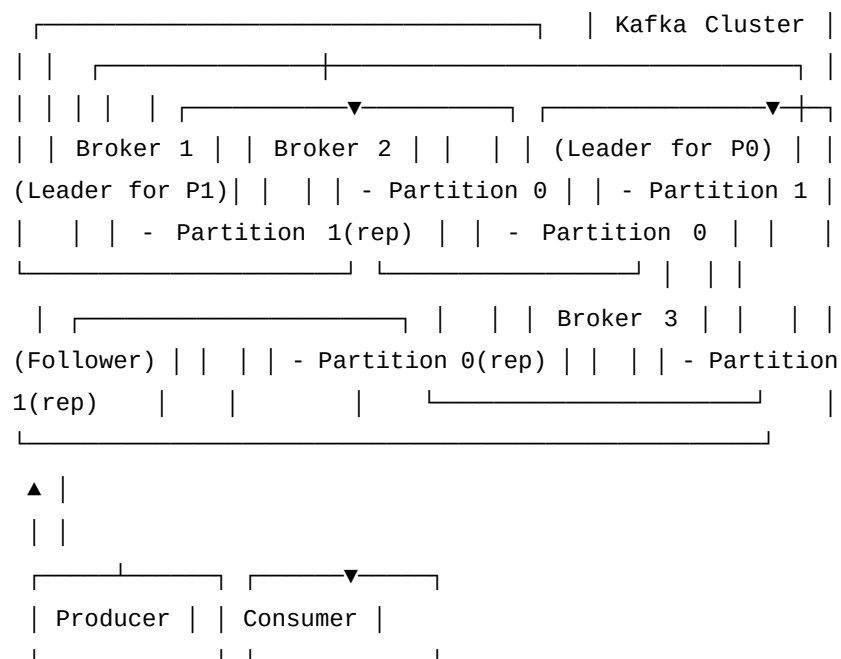
Distributed: Kafka runs as a cluster across multiple servers
Persistent: Messages are stored on disk and replicated
Scalable: Can handle trillions of events per day
High Performance: Sub-millisecond latency with high throughput
Fault-Tolerant: Data replication ensures no data loss

Core Components

1. **Events/Messages:** Immutable records of something that happened
2. **Topics:** Categories where events are published
3. **Producers:** Applications that publish events to topics
4. **Consumers:** Applications that subscribe to topics and process events
5. **Brokers:** Kafka servers that store data and serve clients
6. **ZooKeeper/KRaft:** Coordination service for cluster management

Kafka Architecture

Cluster Architecture



Broker

A Kafka server that:

- Receives messages from producers
- Assigns offsets to messages
- Commits messages to storage
- Serves fetch requests from consumers

Participates in replication

Controller

One broker in the cluster acts as the **Controller**:

Manages partition leadership

Handles broker failures

Coordinates partition reassignments

Elected through ZooKeeper/KRaft

Topics and Partitions

Topics

A **topic** is a named feed or category of messages. Think of it as a table in a database or a folder in a filesystem.

Characteristics:

Multi-subscriber: Multiple consumers can read from the same topic

Immutable: Once written, messages cannot be modified

Configurable retention: Messages can be kept for a specified time or size

Ordered within partitions: Messages maintain order within a single partition

Partitions

Each topic is divided into **partitions** for:

Scalability: Distribute data across multiple brokers

Parallelism: Multiple consumers can read simultaneously

Ordering: Guarantee message order within a partition

Topic: user-events (3 partitions)

Partition 0: [msg0] [msg3] [msg6] [msg9] ...

Partition 1: [msg1] [msg4] [msg7] [msg10] ...

Partition 2: [msg2] [msg5] [msg8] [msg11] ...

↑ Offsets: 0, 1, 2, 3, ...

Partition Key

Producers can specify a **key** for each message:

Messages with the same key go to the same partition

Ensures ordering for related messages

If no key: round-robin or custom partitioning

Example:

Key: "user-123" → Always goes to Partition 1

Key: "user-456" → Always goes to Partition 0

Offsets

An **offset** is a unique sequential ID for each message within a partition:

Starts at 0 for each partition

Increments with each new message

Used by consumers to track position

Immutable and unique within a partition

Producers - Deep Dive

Producer Workflow

1. Producer creates message
2. Serializer converts key/value to bytes
3. Partitioner determines target partition
4. Message added to batch for that partition
5. Batch sent to leader broker
6. Leader writes to log and replicates
7. Acknowledgment sent back to producer

Producer Configuration

Essential Configurations

bootstrap.servers

Initial connection points to Kafka cluster

Format: "host1:9092,host2:9092"

Only need a few brokers (client discovers rest)

key.serializer / value.serializer

Convert objects to bytes

Common: StringSerializer, ByteArraySerializer, JsonSerializer **acks**

(Acknowledgment Level)

`acks=0`: Fire and forget (no acknowledgment)

Fastest, but can lose messages

Use for: Metrics, logs where some loss is acceptable

`acks=1`: Leader acknowledgment (default)

Leader writes to log before acknowledging

Balanced performance and durability

Risk: Message lost if leader fails before replication

`acks=all` or `acks=-1`: All in-sync replicas acknowledge

Highest durability guarantee

Slowest, but no data loss

Use for: Critical data, financial transactions

retries

Number of retry attempts on failure

Default: `INT_MAX` (effectively infinite retries)

Combines with `retry.backoff.ms` for exponential backoff

enable.idempotence

Ensures exactly-once delivery per partition

Prevents duplicate messages on retry

Automatically sets: `acks=all`, `retries=INT_MAX`

Recommendation: Always enable for production

Performance Configurations

batch.size

Maximum bytes for a batch (default: 16KB)

Larger batches = better compression and throughput

Trade-off: Higher latency

linger.ms

Time to wait before sending batch (default: 0)

Allows batching of messages

Example: `linger.ms=10` waits 10ms for more messages

Trade-off: Adds latency for better throughput

compression.type

Compress messages before sending

Options: none, gzip, snappy, lz4, zstd

lz4: Good balance of speed and compression

zstd: Best compression ratio (Kafka 2.1+)

buffer.memory

Total memory for buffering unsent messages (default: 32MB)

Producer blocks if buffer is full

max.in.flight.requests.per.connection

Maximum unacknowledged requests (default: 5)

Set to 1 for strict ordering (with idempotence disabled)

With idempotence: Can be > 1 while maintaining order

Partitioning Strategies

1. Key-based Partitioning (Default)

```
partition = hash(key) % num_partitions
```

Same key → Same partition

Ensures ordering for related events

2. Round-robin (No Key)

Distributes evenly across partitions

No ordering guarantee

3. Custom Partitioner

Implement custom logic

Example: Partition by geographic region

Producer Patterns

Fire-and-Forget

```
// Send without waiting for response
err := writer.WriteMessages(ctx, kafka.Message{
  Key: []byte("key"),
  Value: []byte("value"),
})
```

Synchronous Send

```
// Wait for acknowledgment
```

```
err := writer.WriteMessages(ctx, kafka.Message{
Key: []byte("key"),
Value: []byte("value"),
})
if err != nil {
log.Fatal("failed to write messages:", err) }
```

Asynchronous with Callback

```
// Send and handle result in goroutine
go func() {
err := writer.WriteMessages(ctx, messages...)
if err != nil {
handleError(err)
} else {
handleSuccess()
}
}()
```

Message Structure

Kafka Message
Key : []byte (optional)
Value : []byte (required)
Timestamp : time.Time
Headers : []Header (k-v pairs)
Partition : int (optional)
Offset : int64 (set by broker)

Headers

Metadata about the message

Don't affect partitioning

Use cases:

- Tracing IDs

- Message type/version

- Source system information

- Encryption metadata

Consumers - Deep Dive

Consumer Workflow

1. Consumer subscribes to topic(s)
2. Joins consumer group (if applicable)
3. Receives partition assignment
4. Fetches messages from assigned partitions
5. Processes messages
6. Commits offsets (auto or manual)
7. Repeats from step 4

Consumer Configuration

Essential Configurations

bootstrap.servers

Same as producer

group.id

Consumer group identifier

Consumers with same group.id share partition consumption

Each group maintains separate offsets

key.deserializer / value.deserializer

Convert bytes back to objects

auto.offset.reset

What to do when no initial offset exists

earliest: Start from beginning of topic

latest: Start from end (only new messages)

none: Throw exception if no offset found

enable.auto.commit

Automatically commit offsets periodically

true (default): Convenience, risk of message loss/duplication

false: Manual control, at-least-once or exactly-once semantics

auto.commit.interval.ms

Frequency of auto-commits (default: 5000ms)

Only relevant if enable.auto.commit=true

Performance Configurations

fetch.min.bytes

Minimum data broker should return (default: 1 byte)

Higher value: Better throughput, higher latency

fetch.max.wait.ms

Maximum time broker waits before responding (default: 500ms)

Works with `fetch.min.bytes`

max.partition.fetch.bytes

Maximum data per partition per fetch (default: 1MB)

Should be larger than largest message

max.poll.records

Maximum records returned in single poll (default: 500)

Lower value: More frequent offset commits, better failure recovery

max.poll.interval.ms

Maximum time between polls (default: 300000ms = 5 minutes)

Consumer kicked out of group if exceeded

Increase for slow processing

session.timeout.ms

Time before consumer considered dead (default: 10000ms)

Heartbeat-based detection

Shorter timeout: Faster failure detection, more false positives

heartbeat.interval.ms

Frequency of heartbeats (default: 3000ms)

Should be $< \text{session.timeout.ms} / 3$

Offset Management

What is an Offset Commit?

Committing an offset means storing the position in Kafka's internal `__consumer_offsets` topic.

Topic: `orders`

Partition 0: `[0][1][2][3][4][5][6][7][8][9]`

↑

Last committed offset: 6

(Next read: 7)

Commit Strategies

1. Auto-commit (At-Most-Once)

```
enable.auto.commit=true  
auto.commit.interval.ms=5000
```

Commits automatically every 5 seconds

Risk: Message loss if consumer crashes after commit but before processing

2. Manual Commit After Batch (At-Least-Once)

```
for {  
  m, err := reader.ReadMessage(ctx)  
  if err != nil {  
    break  
  }  
  process(m)  
  reader.CommitMessages(ctx, m) // Commit after processing }  
}
```

Commit only after successful processing

Risk: Duplicate processing if crash after processing but before commit

Transactional Processing (Exactly-Once)

Uses Kafka transactions

Commit offset within transaction

All or nothing: processing + offset commit

Offset Reset

If consumer starts with no committed offset:

auto.offset.reset=earliest

```
[0][1][2][3][4][5][6][7][8][9]
```

↑

Start here (process all historical messages)

auto.offset.reset=latest

```
[0][1][2][3][4][5][6][7][8][9]
```

↑

Start here (only new messages)

Consumer Rebalancing

Rebalance occurs when:

- Consumer joins group
- Consumer leaves group (graceful or crash)
- Consumer is considered dead (heartbeat timeout)
- New partitions added to topic
- Consumer subscription changes

Rebalance Protocol

1. Consumer stops fetching
2. Commits current offsets
3. Revokes partition assignments
4. Coordinator assigns new partitions
5. Consumer starts fetching from new assignments

Rebalance Strategies

Range Assignor (Default)

- Assigns partitions per topic
- Can lead to imbalance across topics

RoundRobin Assignor

- Assigns partitions round-robin across all topics
- Better balance

Sticky Assignor

- Minimizes partition movement during rebalance
- Maintains assignments when possible

Cooperative Sticky Rebalancing (Kafka 2.4+)

- Incremental rebalancing
- Doesn't stop all consumers
- Transfers only affected partitions

Consumer Patterns

Single Consumer

```
reader := kafka.NewReader(kafka.ReaderConfig{
    Brokers: []string{"localhost:9092"},
    Topic: "my-topic",
```

```
})
```

Reads from all partitions

Simple, no group coordination

Consumer Group

```
reader := kafka.NewReader(kafka.ReaderConfig{  
  Brokers: []string{"localhost:9092"},  
  GroupID: "my-consumer-group",  
  Topic: "my-topic",  
})
```

Partitions divided among group members

Automatic rebalancing

Parallel processing

Consumer Groups

What is a Consumer Group?

A **consumer group** is a set of consumers that cooperate to consume data from topics. Each partition is consumed by exactly one consumer in the group.

How It Works

Topic: events (4 partitions)

Consumer Group: "app-processors"

Partition 0 —  Consumer 1

Partition 1 —  Consumer 1

Partition 2 —  Consumer 2

Partition 3 —  Consumer 3

Scaling Consumers

Scenario 1: Consumers < Partitions

4 Partitions, 2 Consumers

P0 —  C1



Load balanced
Can add more consumers

Scenario 2: Consumers = Partitions

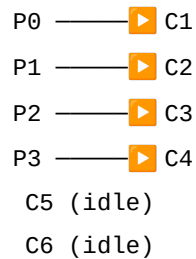
4 Partitions, 4 Consumers



Optimal parallelism
Maximum throughput

Scenario 3: Consumers > Partitions

4 Partitions, 6 Consumers



Excess consumers sit idle
Waste of resources
Rule: Max useful consumers = Number of partitions

Multiple Consumer Groups

Different groups can read the same topic independently:

Topic: user-events

Group "analytics" (Offset: 1000)

Consumer A —▶ Partition 0
Consumer B —▶ Partition 1

Group "notifications" (Offset: 5000)

Consumer C → Partition 0

Consumer D → Partition 1

Each group maintains separate offsets

Independent processing

Fan-out pattern

Group Coordinator

The **Group Coordinator** is a broker that:

Manages consumer group membership

Triggers rebalances

Stores offset commits

Selected based on group.id hash

Message Queue Patterns

1. Point-to-Point (Queue)

Traditional message queue behavior: each message consumed by one

consumer. **Implementation:**

Single consumer group

Multiple consumers in the group

Each message processed once

Producer → Topic → [Consumer Group]

├→ Consumer 1

├→ Consumer 2

└→ Consumer 3

Use Cases:

Task distribution

Job processing

Load balancing work

Example: Order processing system

Orders Topic

├ Order 1 → Worker 1 (processes)

├ Order 2 → Worker 2 (processes)

└─ Order 3 ─▶ Worker 3 (processes)

2. Publish-Subscribe (Pub-Sub)

Each message consumed by all interested consumers.

Implementation:

Multiple consumer groups

Each group receives all messages

Independent processing

└─▶ [Group A] ─▶ Consumer A1, A2 Producer ─▶ Topic
└─▶ [Group B] ─▶ Consumer B1, B2 └─▶ [Group C]
└─▶ Consumer C1, C2

Use Cases:

Event broadcasting

Multiple independent processors

Real-time analytics + archiving

Example: User registration event

UserRegistered Event

└─▶ Email Service (sends welcome email)
└─▶ Analytics Service (tracks metrics)
└─▶ CRM Service (creates contact)
└─▶ Notification Service (sends push notification)

3. Event Sourcing

Store all state changes as a sequence of events.

Characteristics:

Events are immutable

Current state derived from event history

Time travel possible (replay events)

Pattern:

Command ─▶ Event ─▶ Kafka ─▶ Event Processor ─▶ Update
State |

└─▶ Event Store (permanent log) **Example:** Bank account

Events:

- └─ AccountCreated(id: 123, balance: 0)
- └─ MoneyDeposited(id: 123, amount: 100)
- └─ MoneyWithdrawn(id: 123, amount: 30)
- └─ Current Balance = 70 (derived from events)

4. CQRS (Command Query Responsibility Segregation)

Separate read and write models using Kafka.

Commands → Write Model → Events → Kafka → Read Model
Updater └─ Query Database

Benefits:

Optimized read and write paths

Scalability

Different data models for reads/writes

5. Request-Reply Pattern

Implement synchronous-like request-response over Kafka.

Implementation:

1. Client sends request to request-topic with reply-to topic in header
2. Server processes and sends response to reply-to topic
3. Client consumes from reply-topic with correlation ID

Configuration:

Request Message:

```
topic: "requests"
key: "correlation-id-123"
headers: {
  "reply-to": "replies"
  "correlation-id": "correlation-id-123"
}
```

Response Message:

```
topic: "replies"
key: "correlation-id-123"
headers: {
  "correlation-id": "correlation-id-123"
}
```


6. Dead Letter Queue (DLQ)

Handle messages that fail processing.

Pattern:

Main Topic → Consumer
|
├─▶ Success ─▶ Continue
|
└─▶ Failure ─▶ DLQ Topic ─▶ Error Handler

Implementation:

```
for {
    msg, err := reader.ReadMessage(ctx)
    if err != nil {
        break
    }

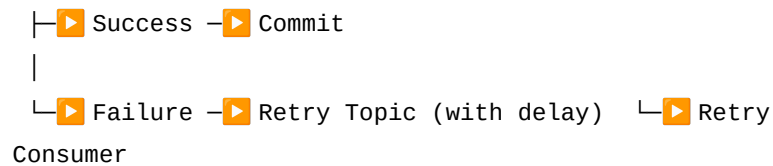
    err = processMessage(msg)
    if err != nil {
        // Send to DLQ
        dlqWriter.WriteMessages(ctx, kafka.Message{
            Key: msg.Key,
            Value: msg.Value,
            Headers: []kafka.Header{
                {Key: "error", Value: []byte(err.Error())}, {Key: "original-topic",
Value: []byte("main-topic")}, {Key: "failed-at", Value:
[]byte(time.Now().String())}, },
            })
    }
    reader.CommitMessages(ctx, msg)
}
```

7. Retry Pattern

Retry failed messages with exponential backoff.

Pattern:

Main Topic → Consumer
|



Implementation:

Create retry topics with different delays
retry-1min, retry-5min, retry-30min
After max retries → DLQ

8. Compaction for Latest State

Use log compaction for maintaining latest state.

Pattern:

Topic (log.cleanup.policy=compact)

Key: user-123

└─ v1: {name: "John", age: 25}

└─ v2: {name: "John", age: 26} ← Kept (latest)

└─ v3: {name: "John Doe", age: 26} ← Kept (latest)

Compaction removes v1 and v2, keeps only v3

Use Cases:

User profiles
Configuration changes
Catalog updates

Replication and Fault Tolerance Replication Basics

Kafka replicates partitions across multiple brokers for fault tolerance.

Topic: orders, Replication Factor: 3

Partition 0:

└─ Leader: Broker 1 (reads/writes)

└─ Follower: Broker 2 (replicates)

└─ Follower: Broker 3 (replicates)

Partition 1:

└─ Leader: Broker 2 (reads/writes)

└─ Follower: Broker 1 (replicates)

└─ Follower: Broker 3 (replicates)

Leader and Followers

Leader:

Handles all reads and writes for a partition

Maintains the authoritative log

Propagates writes to followers

Followers:

Replicate data from leader

Don't serve client requests (in standard config)

Can become leader if current leader fails

In-Sync Replicas (ISR)

ISR is the set of replicas that are fully caught up with the leader.

Partition 0:

└─ Leader: Broker 1 (Offset: 1000) [ISR]

└─ Follower: Broker 2 (Offset: 1000) [ISR] ✓ In-sync

└─ Follower: Broker 3 (Offset: 950) [Not ISR] ✗ Lagging

Replica becomes ISR when:

Caught up within `replica.lag.time.max.ms` (default: 10s)

Replica removed from ISR when:

Falls behind leader by more than time threshold

Not sending fetch requests

Leader Election

When a leader fails:

1. Controller detects failure
2. Selects new leader from ISR
3. Updates metadata

4. Notifies producers/consumers

Unclean Leader Election:

`unclean.leader.election.enable=false` (recommended)

If enabled, non-ISR replica can become leader

Risk: Data loss, but maintains availability

Replication Factor Recommendations

Replication Factor 1:

No fault tolerance

Data loss if broker fails

Use only for: Development, non-critical logs

Replication Factor 2:

Survives 1 broker failure

Minimal for production

Risk: No redundancy during recovery

Replication Factor 3 (Recommended):

Survives 2 broker failures

One replica can be down for maintenance

Good balance of cost vs. safety

Replication Factor > 3:

For critical systems

Higher storage cost

More network bandwidth

Durability Guarantees

Scenario 1: `acks=1`, Replication Factor=3

Producer → Leader writes → ACK immediately

└─→ Follower 1 (async)

└─→ Follower 2 (async)

Risk: Leader fails before replication

Message lost

Scenario 2: `acks=all`, Replication Factor=3, `min.insync.replicas=2`

Producer → Leader writes ─→ Follower 1 writes →

ACK  Follower 2 writes —

Leader + at least 1 follower must acknowledge

High durability

Some performance cost

min.insync.replicas

Minimum number of replicas (including leader) that must acknowledge write:

min.insync.replicas=1:

Only leader needs to acknowledge

Lower durability

min.insync.replicas=2 (Recommended with RF=3):

Leader + 1 follower must acknowledge

Good balance

min.insync.replicas=3 (with RF=3):

All replicas must acknowledge

Maximum durability

Broker failure = no writes (availability reduced)

Formula:

Tolerable failures = Replication Factor - min.insync.replicas

RF=3, min.insync.replicas=2

Tolerable failures = 3 - 2 = 1 broker can fail

Kafka CLI Commands

Cheatsheet Topic Management

Create Topic

Basic topic creation

```
kafka-topics --bootstrap-server localhost:9092 \
--create \
--topic my-topic \
--partitions 3 \
--replication-factor 2
```

```
# With retention settings
kafka-topics --bootstrap-server localhost:9092 \
--create \
--topic my-topic \
--partitions 3 \
--replication-factor 2 \
--config retention.ms=86400000 \
--config segment.ms=3600000
```

```
# With compaction
kafka-topics --bootstrap-server localhost:9092 \
--create \
--topic user-profiles \
--partitions 3 \
--replication-factor 2 \
--config cleanup.policy=compact \
--config min.cleanable.dirty.ratio=0.01
```

List Topics

```
# List all topics
kafka-topics --bootstrap-server localhost:9092 --list

# List with details
kafka-topics --bootstrap-server localhost:9092 --describe

# Describe specific topic
kafka-topics --bootstrap-server localhost:9092 \
--describe \
--topic my-topic
```

Modify Topic

```
# Increase partitions (cannot decrease!)
kafka-topics --bootstrap-server localhost:9092 \
--alter \
--topic my-topic \
--partitions 5

# Change configuration
kafka-configs --bootstrap-server localhost:9092 \
--entity-type topics \
--entity-name my-topic \
--alter \
--add-config retention.ms=172800000
```

```
# Remove configuration
kafka-configs --bootstrap-server localhost:9092 \
--entity-type topics \
--entity-name my-topic \
--alter \
--delete-config retention.ms
```

Delete Topic

```
kafka-topics --bootstrap-server localhost:9092 \
--delete \
--topic my-topic
```

Producer Commands

Console Producer

```
# Simple producer
kafka-console-producer --bootstrap-server localhost:9092 \
--topic my-topic
```

```
# With keys
kafka-console-producer --bootstrap-server localhost:9092 \
--topic my-topic \
--property "parse.key=true" \
--property "key.separator=:"
```

```
# Example input:
```

```
# key1:value1
```

```
# key2:value2
```

```
# With headers
```

```
kafka-console-producer --bootstrap-server localhost:9092 \
--topic my-topic \
--property "parse.headers=true" \
--property "headers.delimiter=," \
--property "headers.key.separator=:"
```

```
# Example input:
```

```
# header1:value1,header2:value2 message-body
```

```
# From file
```

```
cat messages.txt | kafka-console-producer \
--bootstrap-server localhost:9092 \
```

```
--topic my-topic
```

Performance Testing

```
# Producer performance test
```

```
kafka-producer-perf-test --topic my-topic \  
  --num-records 1000000 \  
  --record-size 1024 \  
  --throughput -1 \  
  --producer-props bootstrap.servers=localhost:9092 \  
acks=all
```

```
# Results show:
```

```
# - Records/sec
```

```
# - MB/sec
```

```
# - Average latency
```

```
# - Max latency
```

```
# - Percentiles (50th, 95th, 99th, 99.9th)
```

Consumer Commands

Console Consumer

```
# Consume from latest
```

```
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic
```

```
# Consume from beginning
```

```
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--from-beginning
```

```
# With consumer group
```

```
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--group my-consumer-group
```

```
# Show keys and values
```

```
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--property print.key=true \  
--property key.separator=":"
```

```
# Show headers, keys, values, and timestamps kafka-
```

```
console-consumer --bootstrap-server localhost:9092 \ --  
topic my-topic \
```



```
--property print.timestamp=true \  
--property print.key=true \  
--property print.value=true \  
--property print.headers=true  
  
# Consume specific partition  
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--partition 0  
  
# Consume with offset  
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--partition 0 \  
--offset 100  
  
# Max messages  
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic my-topic \  
--max-messages 10 \  
--from-beginning
```

Performance Testing

```
# Consumer performance test  
kafka-consumer-perf-test --topic my-topic \  
--bootstrap-server localhost:9092 \  
--messages 1000000 \  
--threads 1  
  
# With consumer group  
kafka-consumer-perf-test --topic my-topic \  
--bootstrap-server localhost:9092 \  
--messages 1000000 \  
--group perf-test-group
```

Consumer Group Management

List Consumer Groups

```
# List all consumer groups  
kafka-consumer-groups --bootstrap-server localhost:9092 --list
```

```
# Describe specific group
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-consumer-group \
--describe
```

```
# Output shows:
```

```
# - GROUP, TOPIC, PARTITION
```

```
# - CURRENT-OFFSET, LOG-END-OFFSET, LAG
```

```
# - CONSUMER-ID, HOST, CLIENT-ID
```

Reset Offsets

```
# Reset to earliest
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-consumer-group \
--topic my-topic \
--reset-offsets \
--to-earliest \
--execute
```

```
# Reset to latest
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-consumer-group \
--topic my-topic \
--reset-offsets \
--to-latest \
--execute
```

```
# Reset to specific offset
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-consumer-group \
--topic my-topic \
--reset-offsets \
--to-offset 1000 \
--execute
```

```
# Reset by duration (go back 2 hours)
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-consumer-group \
--topic my-topic \
--reset-offsets \
--by-duration PT2H \
--execute
```

```
# Reset to datetime
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \  
--group my-consumer-group \  
--topic my-topic \  
--reset-offsets \  
--to-datetime 2024-01-01T00:00:00.000 \  
--execute
```

Shift offset by amount (+100 or -100)

```
kafka-consumer-groups --bootstrap-server localhost:9092 \  
--group my-consumer-group \  
--topic my-topic \  
--reset-offsets \  
--shift-by -100 \  
--execute
```

Dry run (preview without executing)

```
kafka-consumer-groups --bootstrap-server localhost:9092 \  
--group my-consumer-group \  
--topic my-topic \  
--reset-offsets \  
--to-earliest \  
--dry-run
```

Delete Consumer Group

Delete consumer group (must be empty/inactive)

```
kafka-consumer-groups --bootstrap-server localhost:9092 \  
--group my-consumer-group \  
--delete
```

Broker and Cluster Management

Describe Cluster

Cluster information

```
kafka-broker-api-versions --bootstrap-server localhost:9092
```

Broker configs

```
kafka-configs --bootstrap-server localhost:9092 \  
--entity-type brokers \  
--entity-name 0 \  
--describe
```

Log Directories

```
# Check log directories
kafka-log-dirs --bootstrap-server localhost:9092 \
  --describe \
  --broker-list 0,1,2
```

Partition Management

Preferred Replica Election

```
# Trigger preferred replica election
kafka-leader-election --bootstrap-server localhost:9092 \
  --election-type preferred \
  --all-topic-partitions
```

Reassign Partitions

```
# Generate reassignment plan
kafka-reassign-partitions --bootstrap-server localhost:9092 \
  --topics-to-move-json-file topics.json \
  --broker-list "0,1,2" \
  --generate

# Execute reassignment
kafka-reassign-partitions --bootstrap-server localhost:9092 \
  --reassignment-json-file reassignment.json \
  --execute

# Verify reassignment
kafka-reassign-partitions --bootstrap-server localhost:9092 \
  --reassignment-json-file reassignment.json \
  --verify
```

ACL Management (Security)

```
# Add ACL
kafka-acls --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:alice \
  --operation Read \
  --topic my-topic

# List ACLs
```

```
kafka-acls --bootstrap-server localhost:9092 \  
--list \  
--topic my-topic
```

```
# Remove ACL
```

```
kafka-acls --bootstrap-server localhost:9092 \  
--remove \  
--allow-principal User:alice \  
--operation Read \  
--topic my-topic
```

Debugging Commands

Get Offsets

```
# Get earliest offset
```

```
kafka-get-offsets --bootstrap-server localhost:9092 \  
--topic my-topic \  
--time -2
```

```
# Get latest offset
```

```
kafka-get-offsets --bootstrap-server localhost:9092 \  
--topic my-topic \  
--time -1
```

```
# Get offset at specific timestamp
```

```
kafka-get-offsets --bootstrap-server localhost:9092 \  
--topic my-topic \  
--time 1609459200000
```

Dump Log Segments

```
# Dump log segment (on broker machine)
```

```
kafka-dump-log --files /var/lib/kafka/data/my-topic-0/00000000000000000000.log \  
--print-data-log
```

```
# Verify index
```

```
kafka-dump-log --files /var/lib/kafka/data/my-topic-0/00000000000000000000.index \  
--verify-index-only
```

Metadata

```
# Get metadata
```

```
kafka-metadata --bootstrap-server localhost:9092 \  

```

```
--topic my-topic
```

```
# Includes:
```

```
# - Broker information
```

```
# - Topic configuration
```

```
# - Partition assignments
```

```
# - Leader/replica information
```

Common Troubleshooting Commands

```
# Check consumer lag
```

```
kafka-consumer-groups --bootstrap-server localhost:9092 \
```

```
--group my-consumer-group \
```

```
--describe | grep LAG
```

```
# Find under-replicated partitions
```

```
kafka-topics --bootstrap-server localhost:9092 \
```

```
--describe \
```

```
--under-replicated-partitions
```

```
# Find unavailable partitions
```

```
kafka-topics --bootstrap-server localhost:9092 \
```

```
--describe \
```

```
--unavailable-partitions
```

```
# Check broker logs
```

```
tail -f /var/log/kafka/server.log | grep ERROR
```

```
# Monitor consumer group lag over time
```

```
watch -n 1 'kafka-consumer-groups --bootstrap-server localhost:9092 \
```

```
--group my-consumer-group --describe'
```

Performance Tuning

Producer Tuning

Throughput Optimization

```
# High throughput configuration
```

```
batch.size=32768 # 32KB batches
```

```
linger.ms=10 # Wait 10ms for batching compression.type=lz4 #
```

```
Fast compression buffer.memory=67108864 # 64MB buffer
```

```
acks=1 # Leader ack only
max.in.flight.requests.per.connection=5
```

Latency Optimization

```
# Low latency configuration
batch.size=0 # No batching linger.ms=0 # Send immediately
compression.type=none # No compression overhead acks=1 #
Balanced durability max.in.flight.requests.per.connection=1
```

Durability Optimization

```
# Maximum durability configuration
acks=all # All ISR must ack enable.idempotence=true #
Exactly-once semantics
max.in.flight.requests.per.connection=5
retries=2147483647 # Infinite retries
compression.type=lz4 # Good compression
```

Consumer Tuning

Throughput Optimization

```
# High throughput configuration
fetch.min.bytes=50000 # Wait for 50KB
fetch.max.wait.ms=500 # Or 500ms timeout
max.partition.fetch.bytes=1048576 # 1MB per partition
max.poll.records=500 # Process 500 records
```

Latency Optimization

```
# Low latency configuration
fetch.min.bytes=1 # Don't wait for data
fetch.max.wait.ms=100 # Quick timeout
max.partition.fetch.bytes=1048576 # Standard size
max.poll.records=100 # Smaller batches
```

Broker Tuning

Operating System

```
# File descriptors
ulimit -n 100000
```

```
# Disable swap
swapoff -a

# VM settings
vm.swappiness=1
vm.dirty_ratio=80
vm.dirty_background_ratio=5

# Network settings
net.core.wmem_default=131072
net.core.rmem_default=131072
net.core.wmem_max=2097152
net.core.rmem_max=2097152
```

Broker Configuration

```
# Performance settings
num.network.threads=8 # Network threads num.io.threads=16
# I/O threads socket.send.buffer.bytes=102400 # 100KB send
buffer socket.receive.buffer.bytes=102400 # 100KB receive
buffer socket.request.max.bytes=104857600 # 100MB max
request

# Log settings
num.recovery.threads.per.data.dir=4
log.flush.interval.messages=10000
log.flush.interval.ms=1000

# Replication settings
num.replica.fetchers=4 # Replica fetcher threads
replica.fetch.max.bytes=1048576 # 1MB per fetch
```

Monitoring Key Metrics

Producer Metrics

record-send-rate: Records sent per second
request-latency-avg: Average request latency
record-error-rate: Failed records per second
buffer-available-bytes: Available buffer memory

Consumer Metrics

records-consumed-rate: Records consumed per second

fetch-latency-avg: Average fetch latency
records-lag-max: Maximum consumer lag
commit-latency-avg: Average commit latency

Broker Metrics

BytesInPerSec: Incoming byte rate
BytesOutPerSec: Outgoing byte rate
MessagesInPerSec: Incoming message rate
RequestsPerSec: Request rate
UnderReplicatedPartitions: Partitions not fully replicated
OfflinePartitionsCount: Unavailable partitions

Best Practices

Topic Design

Naming Conventions

Use consistent naming

orders.created

orders.updated

orders.cancelled

Environment prefix

prod.orders.created

dev.orders.created

Domain grouping

ecommerce.orders.created

ecommerce.payments.processed

Partition Count

Calculate partitions

Target throughput: 100 MB/s

Single partition throughput: 10 MB/s

Required partitions: $100/10 = 10$ partitions

Rule of thumb

Partitions = max(

Target_Throughput / Partition_Throughput,

Expected_Consumer_Count

)

```
# Over-partition for future growth
partitions = calculated_partitions * 1.5
```

Retention Policy

```
# Time-based retention
retention.ms=604800000 # 7 days

# Size-based retention
retention.bytes=1073741824 # 1GB per partition

# Compaction for state
cleanup.policy=compact
min.cleanable.dirty.ratio=0.5
```

Producer Best Practices

1. Always enable idempotence for production

```
enable.idempotence=true
```

2. Use appropriate acks level

Critical data: acks=all

Balanced: acks=1

High throughput, loss acceptable: acks=0

3. Set proper timeouts

```
request.timeout.ms=30000
```

```
delivery.timeout.ms=120000
```

4. Use compression

```
compression.type=snappy # or zstd for better compression
```

5. Handle errors properly

Implement retry logic

Use DLQ for persistent failures

Log errors with context

6. Partition key selection

Use meaningful keys for ordering

Avoid hot partitions (skewed keys)

Consider key cardinality

Consumer Best Practices

1. Use consumer groups for scalability

```
group.id=my-consumer-group
```

2. Handle rebalancing gracefully

Implement `onPartitionsRevoked` callback

Commit offsets before revocation

Clean up resources

3. Control processing rate

```
max.poll.records=100 # Process in smaller batches
```

```
max.poll.interval.ms=300000 # Ensure processing completes
```

4. Commit strategy

Manual commits after processing

Commit in batches for throughput

Use `commitSync()` for critical data

5. Error handling

Catch exceptions per message

Skip or retry failed messages

Use DLQ for poison pills

6. Idempotent processing

Design consumers to handle duplicates

Use unique message IDs

Implement deduplication logic

Operational Best Practices

1. Monitoring

Set up alerts for lag, under-replicated partitions

Monitor broker health (CPU, disk, network)

Track consumer group status

2. Security

- Enable SSL/TLS for encryption
- Use SASL for authentication
- Implement ACLs for authorization

3. Backup and Recovery

- MirrorMaker for cross-cluster replication
- Regular backup of ZooKeeper/KRaft metadata
- Test recovery procedures

4. Capacity Planning

- Monitor disk usage per broker
- Plan for peak traffic (2-3x average)
- Consider retention vs. storage

5. Upgrades

- Test in non-production first
- Rolling upgrades (one broker at a time)
- Keep clients compatible with brokers

Troubleshooting Guide

Common Issues

1. High Consumer Lag

Symptoms:

- Consumer offset far behind latest offset
- Processing delay increases over time

Causes:

- Slow message processing
- Insufficient consumer instances
- Network issues

Solutions:

```
# Check lag
kafka-consumer-groups --bootstrap-server localhost:9092 \
--group my-group --describe

# Solutions:
# 1. Add more consumers (up to partition count)
```

- # 2. Optimize processing logic
- # 3. Increase max.poll.records
- # 4. Use parallel processing within consumer

2. Under-Replicated Partitions

Symptoms:

ISR count < replication factor
Broker logs show replication errors

Causes:

Broker down or slow
Network issues
Disk I/O problems

Solutions:

- # Check under-replicated partitions
- kafka-topics --bootstrap-server localhost:9092 \
- describe --under-replicated-partitions
- # Solutions:
- # 1. Check broker health
- # 2. Check disk space and I/O
- # 3. Increase replica.fetch.max.bytes if messages are large
- # 4. Check network connectivity

3. Producer Timeout Errors

Symptoms:

TimeoutException from producer
Messages not reaching Kafka

Causes:

Network latency
Broker overloaded
Wrong broker address

Solutions:

```
// Increase timeout
writer := &kafka.Writer{
  Addr: kafka.TCP("localhost:9092"),
  Topic: "my-topic",
```

```
WriteTimeout: 30 * time.Second, // Increase timeout
ReadTimeout: 30 * time.Second,
}
```

```
// Check connectivity
// 1. Verify broker addresses
// 2. Test network connectivity
// 3. Check broker logs for errors
```

4. Consumer Rebalancing Too Frequently

Symptoms:

Frequent "Rebalance in progress" logs
Consumer not making progress
Partitions constantly reassigned

Causes:

Processing takes longer than `max.poll.interval.ms`
Heartbeat timeout too short
Consumer crashes

Solutions:

```
// Increase timeouts
reader := kafka.NewReader(kafka.ReaderConfig{
  Brokers: []string{"localhost:9092"},
  GroupID: "my-group",
  Topic: "my-topic",
  MaxWait: 10 * time.Second, // Increase poll interval
  SessionTimeout: 30 * time.Second, // Increase session timeout
  HeartbeatInterval: 3 * time.Second,
})

// Solutions:
// 1. Reduce max.poll.records
// 2. Optimize processing time
// 3. Increase max.poll.interval.ms
// 4. Fix consumer code errors
```

5. Message Loss

Symptoms:

Messages produced but not consumed
Gaps in offset sequence

Causes:

- acks=0 or acks=1 with broker failure
- Consumer commits before processing
- Log retention expired

Solutions:

```
// Use strong durability
writer := &kafka.Writer{
    Addr: kafka.TCP("localhost:9092"), Topic: "my-topic",
    RequiredAcks: kafka.RequireAll, // acks=all Idempotent: true, //
    Enable idempotence }

// Manual commit after processing
msg, err := reader.ReadMessage(ctx)
process(msg)
reader.CommitMessages(ctx, msg) // Commit after processing

// Check retention
// 1. Verify retention.ms is sufficient
// 2. Check disk space on brokers
```

6. Duplicate Messages

Symptoms:

- Same message processed multiple times
- Duplicate records in downstream systems

Causes:

- Consumer commits after processing fails
- Producer retries
- Rebalancing during processing

Solutions:

```
// Enable producer idempotence
writer := &kafka.Writer{
    Idempotent: true,
}

// Idempotent consumer processing
func processMessage(msg kafka.Message) {
    messageID := string(msg.Key)

    // Check if already processed
    if isProcessed(messageID) {
```

```

return // Skip duplicate
}

// Process message
process(msg)

// Mark as processed
markProcessed(messageID)

// Commit offset
reader.CommitMessages(ctx, msg)
}

```

7. Out of Memory Errors

Symptoms:

Producer/Consumer crashes with OOM
 High memory usage

Causes:

Large message batches
 Too many in-flight requests
 Large buffer sizes

Solutions:

```

// Limit memory usage
writer := &kafka.Writer{
    BatchSize: 100, // Smaller batches
    BatchBytes: 1024 * 1024, // 1MB batch size
    MaxAttempts: 3,
}

reader := kafka.NewReader(kafka.ReaderConfig{
    MinBytes: 10e3, // 10KB
    MaxBytes: 10e6, // 10MB
})

// Solutions:
// 1. Reduce batch.size and buffer.memory
// 2. Limit max.poll.records
// 3. Process messages in smaller chunks
// 4. Monitor memory usage

```

8. Slow Consumer Performance

Symptoms:

Low message consumption rate
High processing latency

Causes:

Sequential processing
Network latency
Heavy computation per message

Solutions:

```
// Parallel processing
func consume() {
    semaphore := make(chan struct{}, 10) // Limit concurrency
    for {
        msg, err := reader.ReadMessage(ctx)
        if err != nil {
            break
        }

        semaphore <- struct{}{}
        go func(m kafka.Message) {
            defer func() { <-semaphore }()
            process(m)
            reader.CommitMessages(ctx, m)
        }(msg)
    }
}

// Solutions:
// 1. Use goroutines for parallel processing
// 2. Increase fetch.min.bytes
// 3. Add more consumer instances
// 4. Optimize processing logic
```

Debugging Checklist

Producer Issues:

Check broker connectivity
Verify topic exists
Check broker logs for errors
Validate message format
Review producer configuration
Check network latency

Consumer Issues:

- Verify consumer group is active
- Check partition assignments
- Review consumer lag
- Validate offset commits
- Check for rebalancing
- Review error logs

Broker Issues:

- Check disk space
- Monitor CPU and memory
- Review broker logs
- Check under-replicated partitions
- Verify ZooKeeper/KRaft connectivity
- Check network configuration

Kafka Configuration Reference

Critical Producer Configurations

Configuration	Default	Description	Recommendation
---------------	---------	-------------	----------------

<code>bootstrap.servers</code>		Initial broker	
--------------------------------	--	----------------	--

<code>connections</code>	At least 2 brokers		
--------------------------	--------------------	--	--

<code>acks</code>	1	Acknowledgment level	all for critical data
-------------------	---	----------------------	-----------------------

<code>retries</code>	MAX_INT	Retry	
----------------------	---------	-------	--

<code>enable.idempotence</code>	false	Exactly-once semantics	true for production
---------------------------------	-------	------------------------	---------------------

<code>max.in.flight.requests</code>	5	Unacknowledged requests	5 with idempotence, 1 without
-------------------------------------	---	-------------------------	-------------------------------

<code>compression.type</code>	none	Compression algorithm	lz4 or zstd
-------------------------------	------	-----------------------	-------------

<code>batch.size</code>	16384		
-------------------------	-------	--	--

Batch size in bytes	16KB-32KB	<code>linger.ms</code>	0
---------------------	-----------	------------------------	---

Batch wait time	5-10ms for throughput	<code>buffer.memory</code>	33554432
-----------------	-----------------------	----------------------------	----------

Producer buffer memory	32MB-64MB	<code>request.timeout.ms</code>	30000
------------------------	-----------	---------------------------------	-------

Critical Consumer Configurations

Configuration	Default	Description	Recommendation
---------------	---------	-------------	----------------

<code>bootstrap.servers</code>	-	Initial broker connections	At least 2 brokers
--------------------------------	---	----------------------------	--------------------

<code>group.id</code>	-	Consumer group identifier	
-----------------------	---	---------------------------	--

Required for groups `enable.auto.commit` true Auto commit offsets false for control
`auto.offset.reset` latest Reset behavior earliest or latest `max.poll.records` 500
 Records per poll 100-500 `max.poll.interval.ms` 300000 Max time between polls
 5min, adjust for processing `session.timeout.ms` 10000 Session timeout 10s-30s
`heartbeat.interval.ms` 3000 Heartbeat frequency < session.timeout / 3
`fetch.min.bytes` 1 Minimum fetch size 1KB-50KB for throughput `fetch.max.wait.ms`
 500 Max fetch wait time 100ms-500ms

Critical Broker Configurations

Configuration	Default	Description	Recommendation
<code>broker.id</code>	-1	Unique broker ID	
<code>log.dirs</code>	<code>/tmp/kafka-logs</code>	Data directories	Production SSD paths
<code>num.network.threads</code>	3	Network threads	8 for busy brokers
<code>num.io.threads</code>	8	I/O threads	
<code>log.retention.hours</code>	168	Retention time	Based on use case
<code>log.segment.bytes</code>	1073741824	Segment size	1GB default is good
<code>num.partitions</code>	1	Default partitions	3-6 for new topics
<code>default.replication.factor</code>	1	Default replication	3 for production
<code>min.insync.replicas</code>	1	Minimum ISR	2 for production
<code>unclean.leader.election.enable</code>	false	Allow non-ISR	
<code>log.cleanup.policy</code>	delete	Log cleanup policy	delete for durability

Conclusion

This guide covers Kafka from core concepts to advanced patterns. Key takeaways:

- Producers:** Focus on durability (`acks=all`, idempotence) for critical data
- Consumers:** Use consumer groups for scalability, manual commits for control
- Partitions:** Enable parallelism, but partition count should match consumer needs
- Replication:** Use `RF=3` with `min.insync.replicas=2` for production
- Monitoring:** Track lag, under-replicated partitions, and broker health
- Testing:** Use the CLI commands extensively during development

For detailed Go implementation, refer to the code examples and hands-on lab artifacts.